

Phase 4: Deployment - Interview Preparation Guide

Table of Contents

1. [Phase 4 Overview](#)
 2. [Deployment Architecture](#)
 3. [Step-by-Step Deployment](#)
 4. [Errors Encountered & Solutions](#)
 5. [Environment Variables](#)
 6. [CORS Configuration](#)
 7. [Production Lessons Learned](#)
 8. [Interview Questions & Answers](#)
 9. [Deployment Checklist](#)
-

PHASE 4 OVERVIEW

What is Phase 4?

Taking your application from **local development** to **production on the internet**.

Before Phase 4: Only you can access the app locally **After Phase 4:** Anyone can access your live app from anywhere

Goals

✅ Deploy backend to cloud (Railway) ✅ Deploy frontend to cloud (Vercel) ✅ Connect to cloud database (MongoDB Atlas) ✅ Ensure all features work in production ✅
Handle production-specific issues

Tech Stack for Deployment

```
Frontend:
├─ Vercel (host React app)
├─ Environment variables (API URL)
└─ Build optimization

Backend:
├─ Railway (host Node.js server)
├─ Environment variables (API keys, DB URI)
└─ Auto-redeploy on git push

Database:
├─ MongoDB Atlas (cloud database)
├─ Network access whitelisting
└─ Connection string security

Git:
├─ GitHub (source control)
```

- └─ .gitignore (prevent secrets)
- └─ Auto-deploy on push

DEPLOYMENT ARCHITECTURE



STEP-BY-STEP DEPLOYMENT

Step 1: Prepare GitHub Repository

Goal: Push clean code to GitHub without secrets

```
bash
```

```
# Create .gitignore
echo "node_modules/" > .gitignore
echo ".env" >> .gitignore
echo ".env.production" >> .gitignore

# Initialize git
git init
git add .
git commit -m "Initial commit: AI Code Documentation Generator"

# Push to GitHub
git branch -M main
git remote add origin https://github.com/USERNAME/code-doc-generator.git
git push -u origin main
```

Key: Never commit `.env` files with secrets!

Step 2: Set Up MongoDB Atlas

Goal: Cloud database accessible from anywhere

```
1. Create MongoDB Atlas account (atlas.mongodb.com)
2. Create new cluster
3. Create database user:
   - Username: admin
   - Password: StrongPassword123!
4. Whitelist IPs:
   - Add 0.0.0.0/0 (allow from anywhere)
5. Create database: docgen
6. Get connection string:
   mongodb+srv://admin:PASSWORD@cluster0.iqt2fni.mongodb.net/docgen?
   appName=Cluster0
```

Connection String Format:

```
mongodb+srv://USERNAME:PASSWORD@CLUSTER/DATABASE?options
```

Step 3: Deploy Backend to Railway

Goal: Host Node.js API in the cloud

1. Go to railway.app
2. Sign up with GitHub
3. Create new project
4. Deploy from GitHub repo
5. Set root directory: backend
6. Add environment variables:
 - OPENAI_API_KEY=sk-...
 - MONGODB_URI=mongodb+srv://admin:PASSWORD@...
 - NODE_ENV=production
7. Wait for deployment (2-3 minutes)
8. Copy public domain URL

Result: Backend running at <https://your-service.up.railway.app>

Step 4: Deploy Frontend to Vercel

Goal: Host React app in the cloud

1. Go to vercel.com
2. Sign up with GitHub
3. Import repository: code-doc-generator
4. Set root directory: frontend
5. Add environment variables:
 - REACT_APP_API_URL=<https://your-railway-url.up.railway.app>
 - CI=false (to prevent warnings as errors)
6. Wait for deployment (3-5 minutes)
7. Copy deployment URL

Result: Frontend running at <https://code-doc-generator-vert.vercel.app>

ERRORS ENCOUNTERED & SOLUTIONS

Error 1: "MONGODB_URI connection failed: bad auth"

Cause: Wrong MongoDB credentials

Solution:

1. Verify MongoDB username and password are correct
2. Check for special characters in password
3. Special chars need URL encoding:
 - @ = %40
 - ! = %21
 - # = %23
4. Test with `test-mongo.js` before deploying
5. Verify Network Access allows 0.0.0.0/0

Prevention:

```
javascript
```

```
// Test locally first
const mongoose = require('mongoose');
mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log('✓ Connected'))
  .catch(err => console.log('✗ Error:', err.message));
```

Error 2: "GitHub Push Protection - Exposed API Key"

Cause: Committed `.env.production` with OpenAI API key

Solution:

```
bash
```

```
# Remove from git tracking
git rm --cached backend/.env.production
git commit -m "Remove .env files"

# Rewrite history if needed
git reset --hard HEAD~1
git push -f origin main

# Or use GitHub's unblock link to allow secret
```

Prevention:

```
Always check .gitignore BEFORE first commit
.gitignore should include:
- .env
- .env.production
- .env.*.local
```

Interview Point: "I learned the importance of .gitignore. Secrets should never touch version control. Always use environment variables from your host platform."

Error 3: "Build Failed - Root Directory Not Set"

Cause: Railway building from repo root instead of backend folder

Solution:

1. In Railway dashboard
2. Go to Settings
3. Set "Root Directory" to: backend
4. Redeploy

How It Works:

```
Repository structure:
code-doc-generator/
├── backend/           ← Railway should build from here
│   ├── package.json  (has start script)
│   └── server.js
├── frontend/         ← Vercel builds from here
│   ├── package.json
│   └── src/
```

Error 4: "CORS not allowed - Origin mismatch"

Cause: Frontend URL not in backend's CORS whitelist

Solution:

```
javascript

// In backend/server.js
const allowedOrigins = [
  'http://localhost:3000',           // Development
  'https://code-doc-generator-vert.vercel.app' // Production
];

app.use(cors({
  origin: function(origin, callback) {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('CORS not allowed'));
    }
  }
}));
```

How CORS Works:

1. Browser makes request from Vercel domain
2. Request includes Origin header
3. Backend checks if origin is in whitelist
4. If not, blocks request with 403
5. Frontend gets CORS error

Interview Point: "CORS protects against malicious requests. My solution was to explicitly whitelist allowed origins - a security best practice."

Error 5: "ESLint Error - Unused Variable"

Cause: Vercel treats warnings as errors in CI mode

✖ Build Error:
'API_URL' is assigned a value but never used - no-unused-vars

Solution 1: Disable CI Error Mode

In Vercel Build Settings:
Build Command: CI=false npm run build

Solution 2: Use the Variable

```
javascript

// Make sure API_URL is actually used
const API_URL = process.env.REACT_APP_API_URL;
const response = await axios.get(`${API_URL}/api/docs`);
```

Solution 3: Disable ESLint Rule

```
javascript

// eslint-disable-next-line no-unused-vars
const API_URL = process.env.REACT_APP_API_URL;
```

Error 6: "API Returns 404 - Routes Not Found"

Cause: Backend routes not deployed or CORS blocking

Solution:

```
bash

# Test backend directly with curl or Postman
curl https://your-railway-url/api/health
# Should return JSON

# Test individual routes
curl https://your-railway-url/api/docs
curl https://your-railway-url/api/stats
curl -X POST https://your-railway-url/api/generate-docs \
  -H "Content-Type: application/json" \
  -d '{"code": "print(1)", "language": "python"}'
```

If Routes Return 404:

1. Check Railway deployment status
 2. Force redeploy in Railway dashboard
 3. Check logs for runtime errors
 4. Verify environment variables are set
-

Error 7: "Double Slash in API URL - /api/stats"

Cause: `REACT_APP_API_URL` has trailing slash

Before:

```
REACT_APP_API_URL=https://...up.railway.app/  
API call: ${API_URL}/api/docs  
Result: https://...app//api/docs ❌ (double slash!)
```

After:

```
REACT_APP_API_URL=https://...up.railway.app  
API call: ${API_URL}/api/docs  
Result: https://...app/api/docs ✅ (single slash!)
```

Prevention:

```
javascript  
  
// Defensive code  
const API_URL = process.env.REACT_APP_API_URL.replace(/\/$/, '');  
// Now safe regardless of trailing slash
```

Error 8: "Environment Variable Not Loaded in Build"

Cause: Environment variable set after deployment started

Solution:

```
1. Set environment variable in Vercel dashboard  
2. Redeploy (Vercel rebuilds with new variables)  
3. Don't just refresh page (old build is cached)  
  
Key: Environment variables must be set BEFORE build
```

How Variables Work in Build:

```
Build Timeline:  
1. Vercel reads environment variables  
2. React build starts  
3. process.env.REACT_APP_* gets baked into build  
4. Build artifact created (static files)  
5. Deployed to CDN  
  
If you change variable AFTER build, the old value is still in the artifact!
```

ENVIRONMENT VARIABLES

Backend Variables (Railway)

```
env

# API Configuration
OPENAI_API_KEY=sk-your-actual-key-here
NODE_ENV=production

# Database Configuration
MONGODB_URI=mongodb+srv://admin:PASSWORD@cluster0.iqt2fni.mongodb.net/docgen?

# Server Configuration
PORT=5000
```

Security Best Practices:

- ☒ Store in Railway dashboard (not in code)
 - ☒ Use strong passwords (random, 20+ chars)
 - ☒ Rotate API keys regularly
 - ☒ Never log secrets
 - ☒ Use separate keys for dev/production
-

Frontend Variables (Vercel)

```
env

REACT_APP_API_URL=https://code-doc-generator-production.up.railway.app
```

Important Notes:

- ☒ Must start with `REACT_APP_` (React convention)
- ☒ Set in Vercel "Environment Variables" tab
- ☒ Select "Production" environment
- ☒ Redeploy after changing
- ☒ No trailing slash

Verification in Browser Console:

```
javascript

console.log(process.env.REACT_APP_API_URL);
// Shows: https://code-doc-generator-production.up.railway.app
```

CORS CONFIGURATION

What is CORS?

CORS (Cross-Origin Resource Sharing) = Browser security feature

Prevents:

- Malicious websites from accessing your API
- Unauthorized requests from unknown origins

Browser enforces:
Request from vercel.app → railway.app
Browser checks: "Is vercel.app allowed to talk to railway.app?"
Backend must explicitly allow it!

Implementation

Backend (Node.js + Express):

```
javascript

const cors = require('cors');
const app = express();

// Define allowed origins
const allowedOrigins = [
  'http://localhost:3000', // Local dev
  'https://code-doc-generator-vert.vercel.app' // Production
];

// Configure CORS
app.use(cors({
  origin: function(origin, callback) {
    // Allow requests with no origin (mobile apps, Postman, curl)
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('CORS not allowed'));
    }
  },
  credentials: true // Allow cookies
})));
```

How It Works

1. Browser makes request:
Origin: https://vercel.app
2. Browser sends OPTIONS (preflight) to backend
3. Backend responds:

```
Access-Control-Allow-Origin: https://vercel.app
```

4. Browser sees origin matches, allows request

5. Actual GET/POST request proceeds

Testing CORS

```
bash
```

```
# This will fail if CORS not configured
```

```
curl -H "Origin: https://vercel.app" \  
https://railway.app/api/docs
```

```
# Should see in response:
```

```
Access-Control-Allow-Origin: https://vercel.app
```

PRODUCTION LESSONS LEARNED

1. Environment Separation

Development:

```
Frontend: http://localhost:3000
```

```
Backend: http://localhost:5000
```

```
Database: Local MongoDB
```

Production:

```
Frontend: https://vercel.app
```

```
Backend: https://railway.app
```

```
Database: MongoDB Atlas (cloud)
```

Key: Keep configs separate, use environment variables

2. Secrets Management

✖ Bad:

```
javascript
```

```
const API_KEY = "sk-1234567890"; // In code!
```

✔ Good:

```
javascript
```

```
const API_KEY = process.env.OPENAI_API_KEY; // From env
```

Why:

- Secrets never in git history
 - Easy to rotate keys
 - Different keys per environment
 - Team members don't see real secrets
-

3. Build Pipelines

```
Git Push → GitHub
    ↓
Railway detects change
    ↓
Pulls latest code
    ↓
Reads environment variables
    ↓
Rebuilds Docker image
    ↓
Deploys to production
    ↓
Automatic testing & monitoring
```

Benefits:

- No manual deployments
 - Consistent build process
 - Easy rollback if needed
 - Audit trail of changes
-

4. Debugging Production Issues

Problem: Works locally, fails in production

Investigation:

```
javascript
```

```
// 1. Check environment variables
console.log('API_URL:', process.env.REACT_APP_API_URL);

// 2. Check network requests
// DevTools → Network tab → see full URL

// 3. Check backend logs
// Railway dashboard → Deployments → View logs

// 4. Test endpoints with Postman
// Don't rely on frontend to test

// 5. Check CORS headers
// Network tab → Response headers → Access-Control-*
```

5. Performance Optimization

Frontend (Vercel):

- ☒ Automatic code splitting
- ☒ Image optimization
- ☒ CDN delivery
- ☒ Caching headers

Backend (Railway):

- ☒ Environment-based optimizations
 - ☒ Database connection pooling
 - ☒ Request logging
 - ☒ Error monitoring
-

INTERVIEW QUESTIONS & ANSWERS

Deployment Architecture

Q: "Explain your deployment architecture"

A: "I deployed a three-tier architecture:

1. **Frontend (Vercel)**
 - React application hosted on Vercel's CDN
 - Builds automatically on git push
 - Uses REACT_APP_* environment variables
 - Returns static HTML/CSS/JS
2. **Backend (Railway)**
 - Node.js/Express API server
 - Runs containerized on Railway

- Reads environment variables on startup
- Connects to MongoDB Atlas

3. Database (MongoDB Atlas)

- Cloud MongoDB instance
- Network access whitelisted globally
- Connection string stored as environment variable
- Auto-backups enabled

Key insight: Each component scales independently. I can deploy frontend updates without touching backend, and vice versa. "

Error Handling

Q: "What's the most interesting error you encountered during deployment?"

A: " The 'double slash' bug was subtle. My frontend was making requests to:

```
https://...railway.app//api/docs
```

The extra slash broke the routing. The root cause was my environment variable had a trailing slash:

```
REACT_APP_API_URL=https://...railway.app/
```

Then when building the request:

```
${API_URL}/api/docs  
// Result: ...app//api/docs ❌
```

The fix was simple - remove the trailing slash. But it taught me about string concatenation edge cases and the importance of defensive programming:

```
javascript  
  
const API_URL = process.env.REACT_APP_API_URL.replace(/\/$/, '');
```

This is why testing in production with real tools (Postman, Network tab) is crucial. "

Environment Variables

Q: "How do you manage secrets in production?"

A: " I follow these practices:

1. Never commit secrets

- .gitignore prevents accidental commits
- GitHub Secret Scanning caught my exposed API key

2. Use platform-provided secret management

- Railway: Environment Variables dashboard
- Vercel: Environment Variables tab
- Secrets set before build, never in artifacts

3. Rotate keys regularly

- My initial OpenAI key was exposed
- I created a new one and updated it everywhere
- Deleted the old key from OpenAI dashboard

4. Different secrets per environment

- Dev secrets (local/test keys)
- Production secrets (real API keys)
- Staging secrets (for testing)

5. Principle of least privilege

- Each service only gets keys it needs
 - Frontend never gets backend API key
 - Backend never gets sensitive frontend data "
-

CORS

Q: "Explain CORS and why it's important"

A: " CORS (Cross-Origin Resource Sharing) is a browser security feature.

Problem it solves: Without CORS, a malicious website could make requests to my API and steal data or perform actions on behalf of users.

How it works:

1. Browser sees request going to different domain
2. Sends preflight OPTIONS request
3. Backend must explicitly allow that origin
4. If allowed, actual request proceeds

My implementation:

```
javascript
```

```
const allowedOrigins = [
  'http://localhost:3000', // Dev
  'https://code-doc-generator-vert.vercel.app' // Production
];

app.use(cors({
  origin: function(origin, callback) {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('CORS not allowed'));
    }
  }
}));
```

This explicitly allows my frontend, but blocks requests from anywhere else.

Interview insight: CORS is a security feature, not a bug. It forces you to be intentional about who can access your API."

Database Connectivity

Q: "How do you ensure database reliability in production?"

A: "

1. Connection String Security

- Store in environment variables only
- Never hardcode credentials
- Use strong passwords (20+ chars)

2. Network Access

- MongoDB Atlas: Whitelist IPs that need access
- In development: Allow 0.0.0.0/0
- In production: Whitelist specific Railway server IPs

3. Connection Pooling

- Mongoose handles connection pooling
- Prevents connection exhaustion
- Reuses connections for efficiency

4. Error Handling

javascript


```
mongoose.connect(MONGODB_URI)
  .then(() => console.log('✓ DB connected'))
  .catch(err => {
    console.error('✗ DB error:', err);
    // Don't crash - retry connection
  });
```

5. Backup & Recovery

- MongoDB Atlas provides automated backups
- Can restore from point-in-time
- Data is replicated across servers

6. Monitoring

- Check connection status on startup
- Log connection errors
- Monitor query performance "

DEPLOYMENT CHECKLIST

Before Deployment

- ☒ All tests pass locally
- ☒ No console errors in browser
- ☒ Backend API works with Postman
- ☒ Database connection tested locally
- ☒ `.gitignore` includes `.env` files
- ☒ No secrets in code or .git history
- ☒ Environment variables documented

GitHub Setup

- ☒ Repository created and public
- ☒ Code pushed to main branch
- ☒ `.gitignore` configured
- ☒ No `.env` files in git history
- ☒ README present

MongoDB Setup

- ☒ Cluster created
- ☒ Database user created (not admin)
- ☒ Network access allows 0.0.0.0/0
- ☒ Connection string tested locally
- ☒ Connection string format correct

Railway Setup

- ☒ Account created and GitHub connected
- ☒ Project created from GitHub
- ☒ Root directory set to `backend`
- ☒ All environment variables added
- ☒ Deployment successful (green checkmark)
- ☒ Public domain URL copied

Vercel Setup

- ☒ Account created and GitHub connected
- ☒ Project created from GitHub
- ☒ Root directory set to `frontend`
- ☒ Build command: `CI=false npm run build`
- ☒ Environment variables added:
 - `REACT_APP_API_URL=https://...railway.app`
 - **NO trailing slash!**
- ☒ Redeployed after variables added
- ☒ Deployment successful

Testing

- ☒ Health endpoint responds (backend works)
- ☒ All API endpoints work in Postman
- ☒ Frontend loads without errors
- ☒ Generate docs works (end-to-end)
- ☒ Search works
- ☒ History loads
- ☒ No CORS errors in console
- ☒ No 404 errors for API calls

Security

- ☒ No secrets in git history
- ☒ API keys rotated after exposure
- ☒ CORS whitelist configured correctly
- ☒ MongoDB Network Access reviewed
- ☒ Environment variables only in platform dashboards

PRODUCTION DEPLOYMENT METRICS

What We Achieved

Before Phase 4:

- Local only development
- Can't share with others
- No real users
- No uptime monitoring

After Phase 4:

- Live on internet
- Anyone can access
- Scaled to handle traffic
- Auto-redeploy on git push
- Monitoring & logs available
- 99.9% uptime (platform guarantee)

Deployment Success Rate

Deployments attempted: 8

Successful: 8 ✓

Failed: 0

Success rate: 100% (with debugging)

Key: Each error was caught, debugged, and fixed
None required rollback or manual intervention

WHAT TO MENTION IN INTERVIEWS

Strong Points

✓ "I deployed to production using industry-standard platforms" ✓ "I handled CORS security properly" ✓ "I managed environment variables securely" ✓ "I debugged networking issues systematically" ✓ "I set up automated deployments via git push" ✓ "I used MongoDB Atlas for cloud database" ✓ "I configured separate development and production environments"

Growth Areas to Mention

- "Next, I'd add CI/CD testing pipelines"
- "I'd implement monitoring and error tracking (Sentry)"
- "I'd set up load testing for performance"
- "I'd add user authentication (Phase 5)"
- "I'd implement database backups and disaster recovery"

Phase 4 Complete! You've deployed a production-grade application! 🚀